CS184/284A Fall 2026 Homework 2 Write-Up

Names: Team 19

Link to webpage: https://cal-cs184-student.github.io/hw2-19-writeup/
Link to GitHub repository: https://github.com/cal-cs184-student/hw2-19

Overview

In this homework I did Bezier curves and surfaces using de Casteljau's algorithm, and then a bunch of half-edge mesh stuff: vertex normals, edge flips, edge splits, and Loop subdivision. The half-edge part was definitely the hardest because if one pointer is wrong the whole mesh breaks. Drawing out the triangles and labeling every halfedge before coding helped a lot. I also did the extra credit for splitting boundary edges and Loop subdivision on meshes with boundaries.

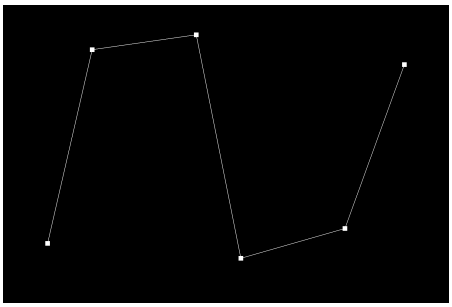
Section I: Bezier Curves and Surfaces

Part 1: Bezier curves with 1D de Casteljau subdivision

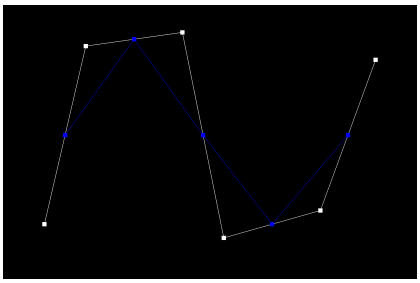
de Casteljau's algorithm is basically just lerping over and over. If you have n control points, you lerp between each pair of neighbors at t, which gives you n - 1 points. Then you do it again to get n - 2 points, and keep going until there's only one point left. That last point is on the curve.

For `evaluateStep` I just loop through the points and push back `(1 - t) * p_i + t * p_(i+1)` for every pair of neighbors. Each call does one level, so pressing E steps down one level at a time.

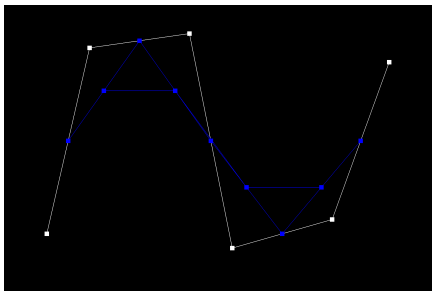
I made my own curve with 6 control points (`bzc/my_curve.bzc`). Here's every level at t = 0.5:



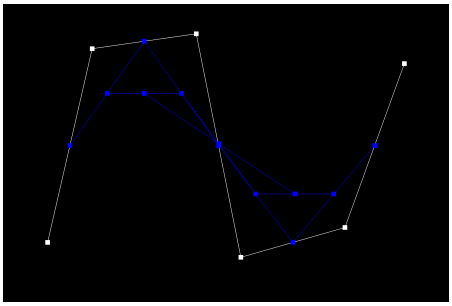
Level 0 (control points)



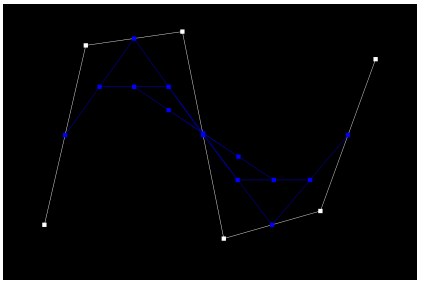
Level 1



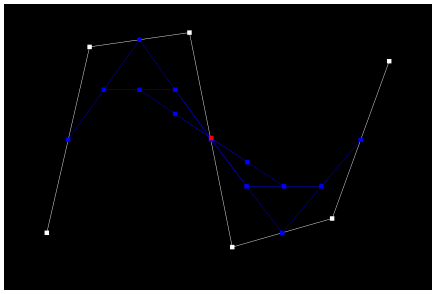
Level 2



Level 3

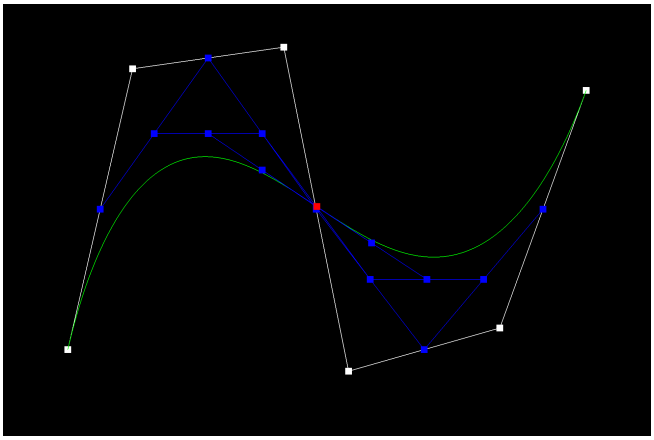


Level 4

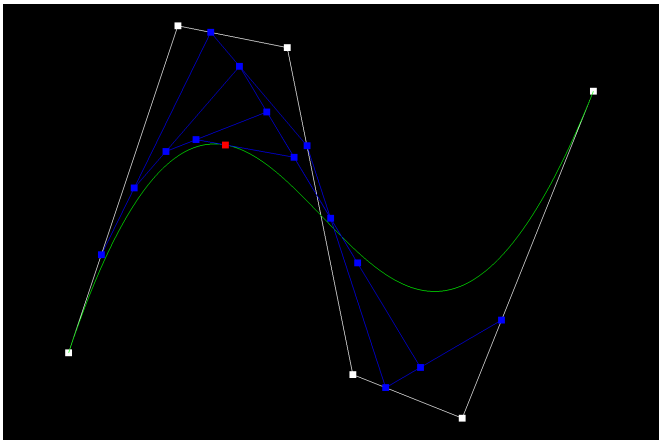


Level 5 (the red point is on the curve)

Here it is with the full curve turned on (C), and then a slightly different curve after I dragged two of the control points around and scrolled t down to 0.3:



With the curve, t = 0.5

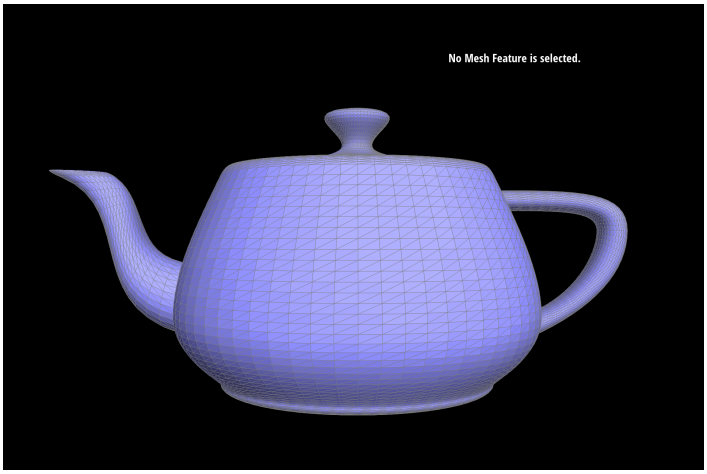


Moved two points, t = 0.3

Part 2: Bezier surfaces with separable 1D de Casteljau

For surfaces you have a 4×4 grid of control points. Each row of the grid is basically a Bezier curve in u. So I run 1D de Casteljau on each of the 4 rows with u, which gives 4 points, and then run it one more time on those 4 points with v. The point that comes out is on the surface.

`evaluateStep` is the same as Part 1 but with 3D points. `evaluate1D` keeps calling `evaluateStep` until there's one point left. `evaluate` does the rows first and then the column.



bez/teapot.bez



Same teapot rotated, with shading on

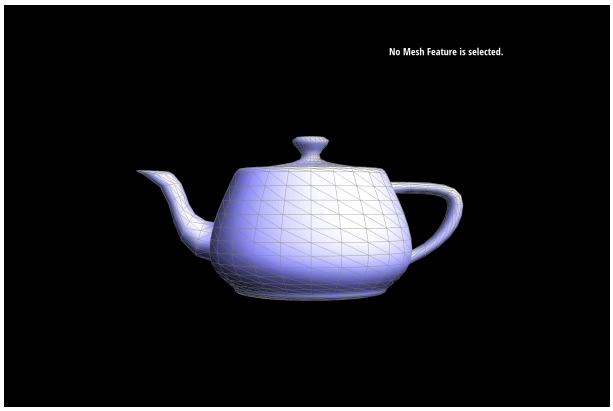
Section II: Triangle Meshes and Half-Edge Data Structure

Part 3: Area-weighted vertex normals

To get the normal at a vertex, I go around the vertex with `h = h->twin()->next()` until I'm back where I started. For every face (skipping the boundary face), I take the cross product of two edges of the triangle. The length of the cross product is 2 times the triangle's area, so just adding them all up already weights each face by its area. At the end I normalize the sum.



dae/teapot.dae, flat shading

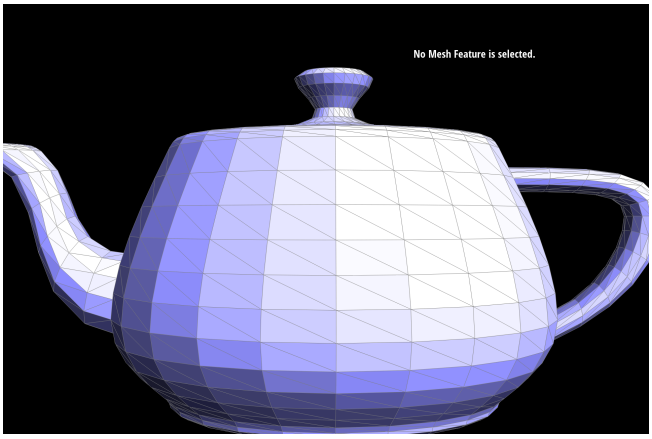


After pressing Q (vertex normals)

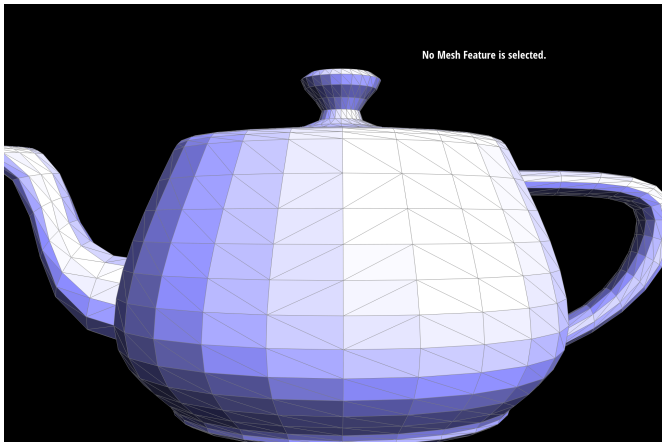
With the vertex normals it looks way smoother and you can't really see the flat bands anymore.

Part 4: Edge flip

For the flip I drew the two triangles and labeled everything: the 6 halfedges, 4 vertices, 2 faces and the edge. After the flip the edge goes between the two other vertices instead. Nothing gets created or deleted, it's just reassigning pointers. I set every pointer with `setNeighbors`, even the ones that don't actually change, so I wouldn't miss any. If the edge is on the boundary it just returns without doing anything.



Before

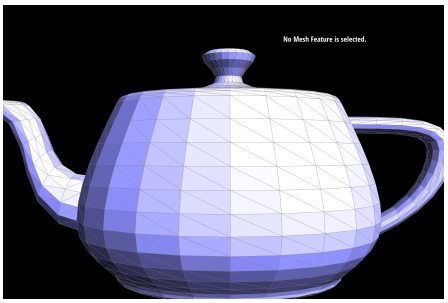


After flipping some edges in the middle

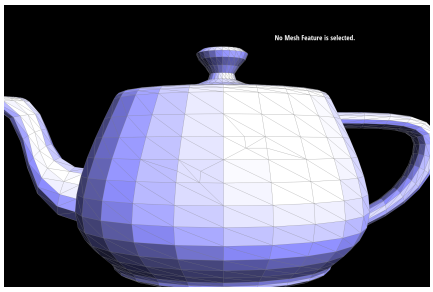
Debugging: it seemed to work at first, but after doing a bunch of random flips on a small mesh it eventually broke. Turns out if a vertex gets flipped down to only 2 triangles, both triangles next to an edge can end up sharing the same third vertex, and flipping that makes an edge that goes from a vertex to itself. I added a check to skip that case and after that it was fine.

Part 5: Edge split

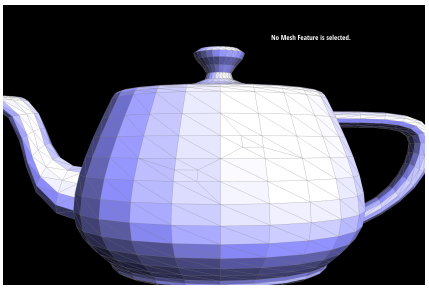
Split is kind of like flip but you have to make new stuff: 1 vertex, 3 edges, 2 faces and 6 halfedges. The new vertex goes at the midpoint of the edge. The old edge turns into one half, a new edge is the other half, and two new edges connect the midpoint to the two opposite vertices. I also set `isNew` on the new vertex and edges here because Part 6 needs it.



Before



After some splits

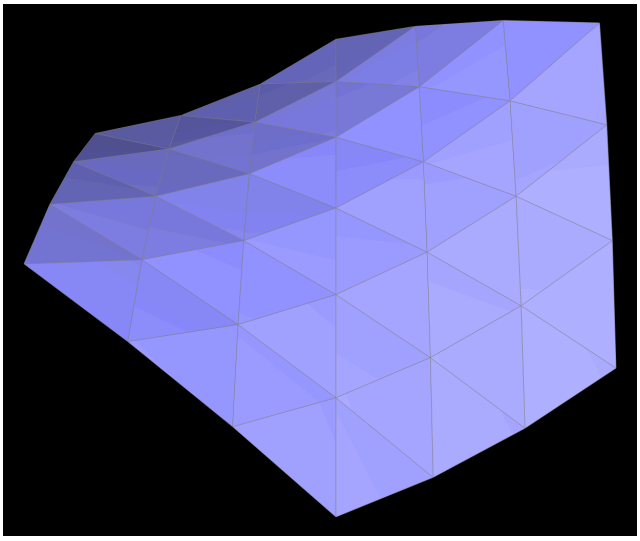


After splits and flips

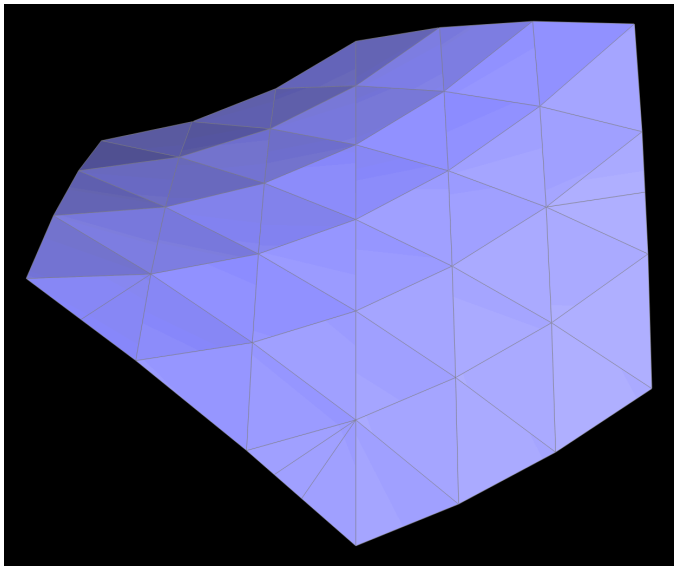
For the splits and flips one, I split all 3 edges of two triangles and then flipped one edge inside each of them, so each one turned into 4 smaller triangles.

Debugging: this one went smoother since I already had the labeling figured out from flip. One thing that confused me was that flipping right after a split sometimes made a weird dark triangle. That happened because the new edge went straight through the midpoint, so the triangle was flat with zero area. The spec says that's ok.

Extra credit (boundary edges): if one side of the edge is the boundary, I only split the real triangle, so it's 1 vertex, 2 edges, 1 face and 4 halfedges, and the new halfedge gets added into the boundary loop. I made a small open mesh to test it (`dae/saddle.dae`).



Before



After splitting some boundary edges

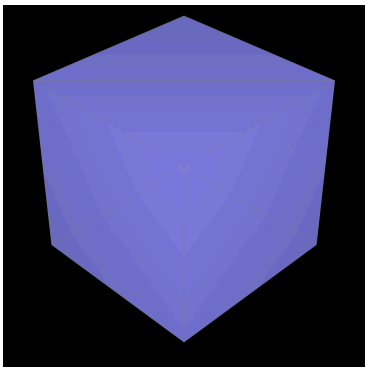
Part 6: Loop subdivision for mesh upsampling

I did it the way the spec recommends:

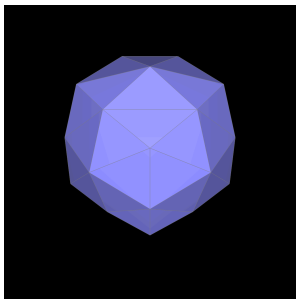
1. Compute the new position for every old vertex with `(1 - n * u) * original_position + u * original_neighbor_position_sum` and store it in `newPosition`.
2. Compute the position for the new vertex on every edge with `3/8 * (A + B) + 1/8 * (C + D)` and store it in the edge's `newPosition`.
3. Split every original edge. I put the original edges in a vector first so I don't keep splitting the new ones forever.
4. Flip every new edge that connects an old vertex and a new vertex.
5. Copy `newPosition` into `position` for every vertex.

To check it I made sure the number of faces went up by 4 times each time I pressed L.

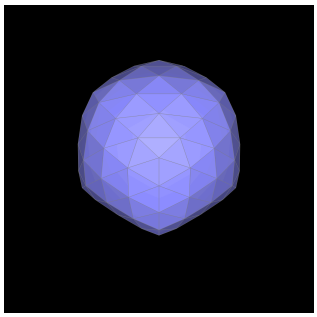
Loop subdivision makes the mesh smoother, but it also shrinks it. Sharp corners and edges get rounded off a lot, and the cube basically turns into a blob:



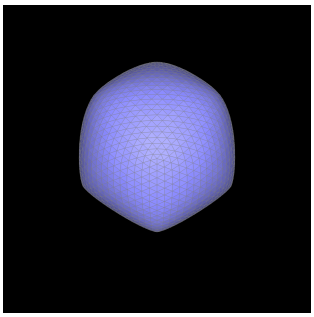
cube.dae



1 time

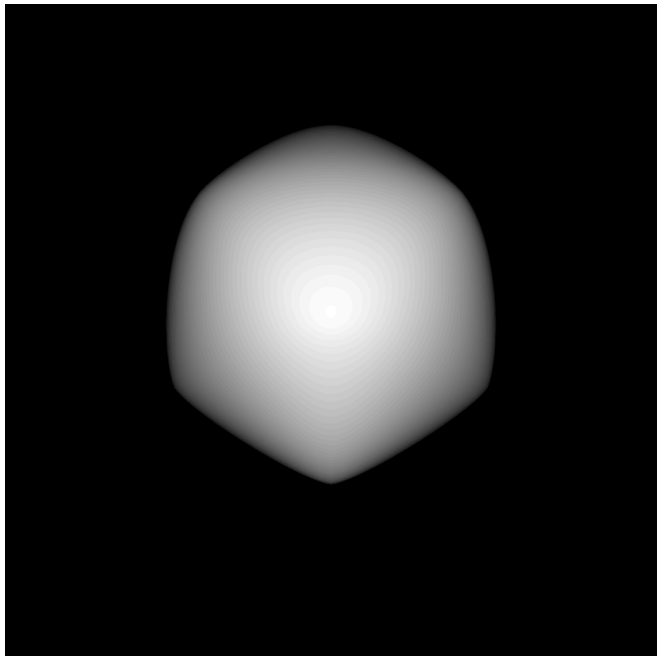


2 times

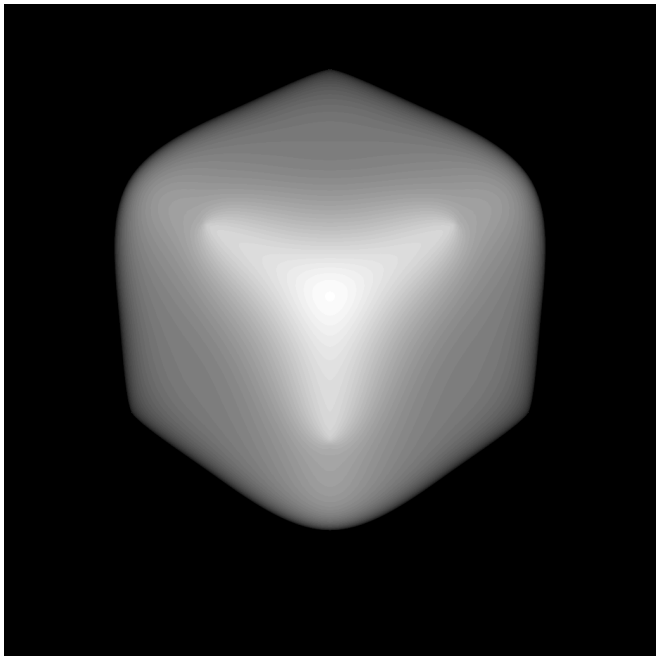


4 times

If you pre-split some edges near the corners and edges, those parts stay sharper because there are more vertices holding the shape there. Here I split the face diagonals and all the cube edges before subdividing, and it stays a lot more cube-like:



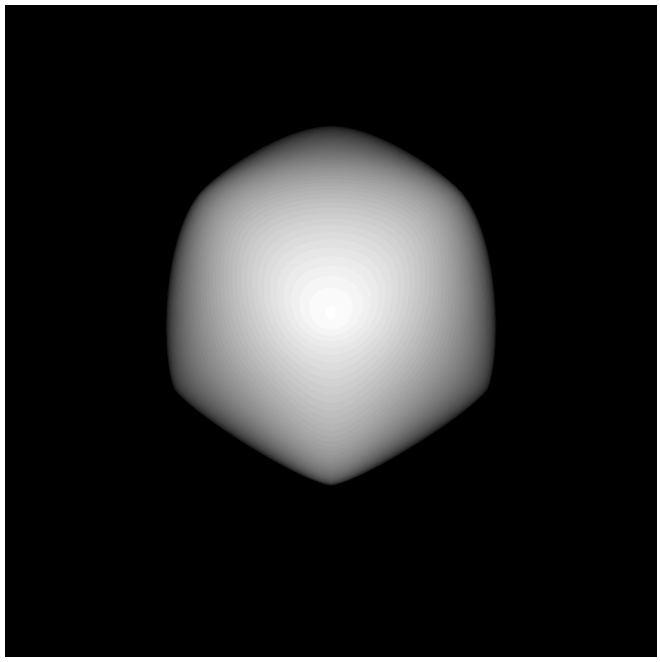
No pre-splitting, 4 times



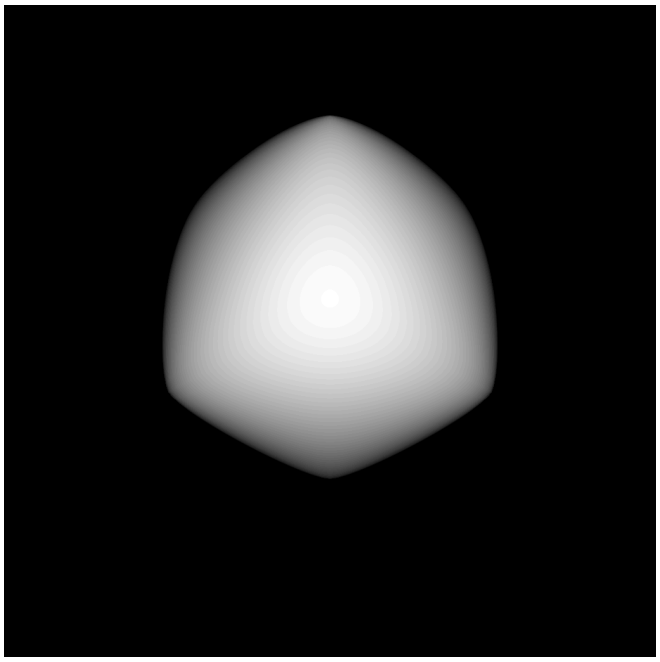
With pre-splitting, 4 times

After a few rounds the cube also looks lopsided. That's because each face of the cube is split into 2 triangles by a diagonal, and the diagonals aren't arranged symmetrically. So some corners have 3 neighbors and some have up to 6. Loop's weights depend on that, so different corners get pulled in by different amounts.

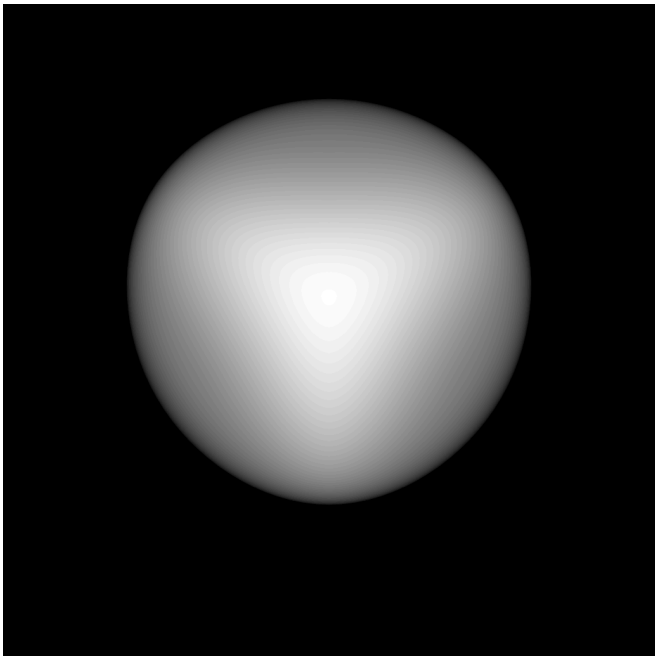
To fix it, I split the diagonal on every face first. Now each face has a vertex in the middle connected to all 4 corners, so every face looks the same and every corner has the same number of neighbors. After that it subdivides symmetrically:



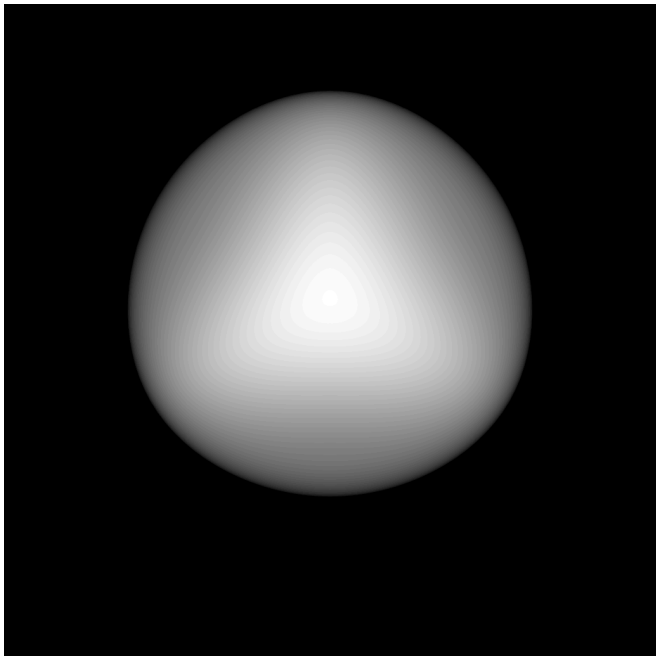
Normal cube, one side



Normal cube, other side (looks different)

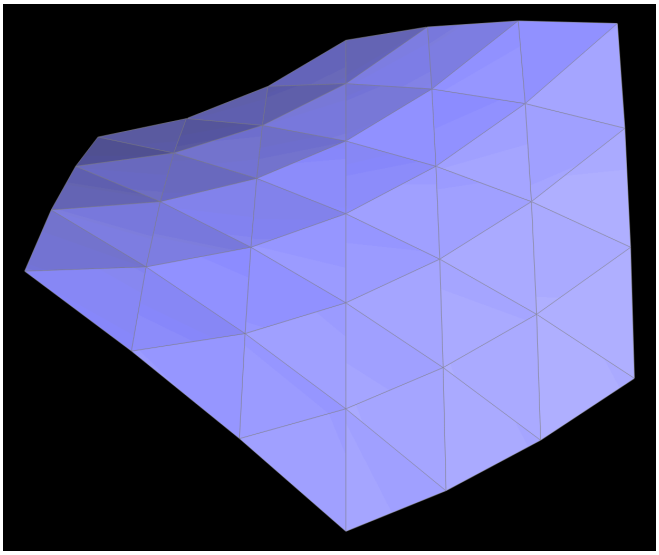


Diagonals split first, one side

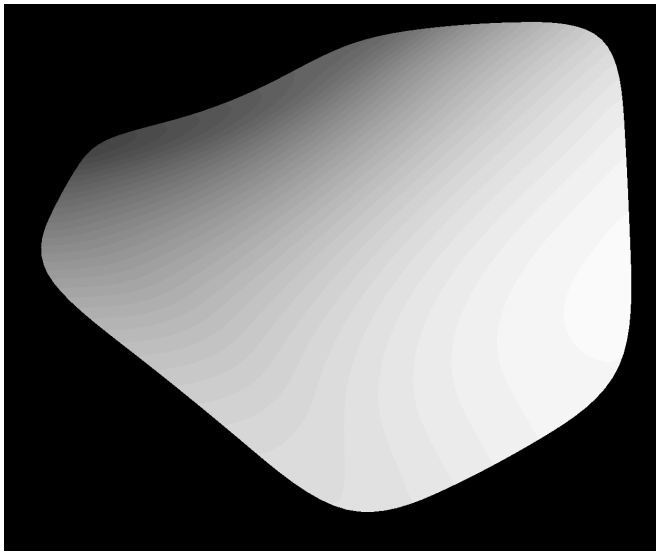


Diagonals split first, other side (same now)

Extra credit (meshes with boundary): boundary vertices use `3/4 * v + 1/8 * (a + b)`, where a and b are its two neighbors along the boundary, and boundary edges just use the midpoint. That keeps the boundary smooth instead of it getting messed up.



Open mesh



After 3 times

Section III: Potential Extra Credit - Art Competition: Model something Creative

I didn't do the art competition.